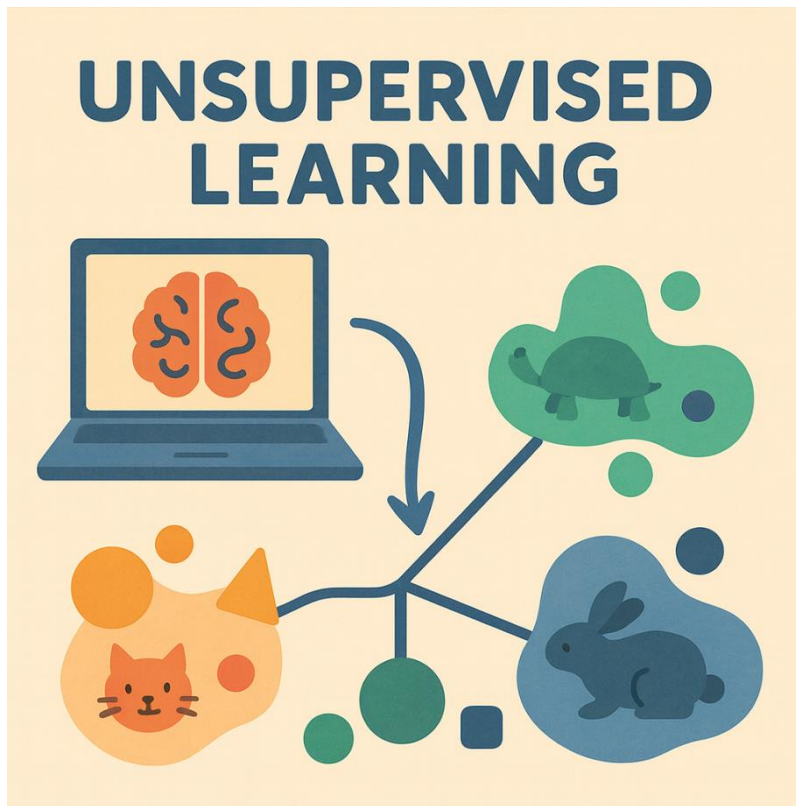




NF3 - MODELS NO SUPERVISATS



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Introducció.....	4
Tipus de problemes que resolen.....	4
Clustering (agrupament)	4
Reducció de dimensionalitat.....	5
Detecció d'anomalies.....	5
<i>Clustering</i> (Agrupament).....	6
K-Means	8
Com funciona?	8
Inicialització dels centroides	9
Número de clústers (k) ?	12
Límits de k-means	15
DBSCAN	16
GMM (Gaussian Mixture Models).....	23
RESUM.....	26
K-Medoids (PAM)	26
HDBSCAN	26
DTW (Dynamic Time Warping)	26
K-Means per seqüències	26
ANNEX	26
Reducció de la dimensionalitat	27
PCA.....	27
t-SNE.....	27
UMAP	27
Autoencoders (introducció molt bàsica).....	27
REFERÈNCIES	27

Introducció

L'**aprenentatge no supervisat** és un tipus d'aprenentatge automàtic (Machine Learning) en què el model ha de **descobrir estructura i patrons ocults dins les dades sense observacions etiquetades**.

En Machine Learning no supervisat, el model no aprèn a predir una resposta, sinó a entendre com estan organitzades les dades.

El model no sap què és correcte o incorrecte. No hi ha una “solució oficial”. Sinó que el model analitza similituds, distàncies, densitats i relacions internes per proposar una manera d'organitzar la informació.

Podem comparar-ho amb deixar que l'ordinador “explori” un conjunt de dades i descobreixi estructures amagades. És com per exemple tenir una caixa de peces de Lega barrejades, però sense cap imatge ni instruccions. La seva feina és agrupar peces similars, detectar peces estranyes, organitzar-les de manera lògica i simplificar el conjunt perquè sigui més manejable.

Tipus de problemes que resolen

Tot i que existeixen molts algorismes diferents, la majoria de tècniques no supervisades es poden agrupar en tres grans famílies segons el tipus de problema que resolen.

Clustering (agrupament)

L'agrupament o **clustering** consisteix a dividir les dades en grups (clústers) de manera que els elements dins d'un mateix grup siguin més semblants entre ells que respecte als elements d'altres grups.

El model **no sap quants grups existeixen ni què representen**: simplement detecta patrons de similitud.

Exemple

Una empresa de comerç electrònic analitza els seus clients a partir de variables com: nombre de compres, import total gastat, temps entre compres, categories preferides,...

Sense cap etiqueta prèvia, un algorisme de *clustering* pot descobrir grups com:

- Clients recurrents de baix import

- Clients ocasionals d'alt valor
- Clients nous
- Clients inactius

Cap d'aquests grups existia abans: el model els ha descobert.

Analogia

És com entrar en una classe nova i, sense parlar amb ningú, agrupar els alumnes segons hàbits observables: qui treballa sol, qui col·labora, qui pregunta molt, qui és més reservat, etc.

Reducció de dimensionalitat

Moltes dades modernes tenen moltes variables (de vegades desenes, centenars o milers). Treballar amb tantes dimensions pot ser difícil d'interpretar, costós a nivell computacional i poc visualitzable.

La reducció de dimensionalitat busca representar les dades amb menys variables, mantenint el màxim d'informació rellevant possible i l'estructura de les dades.

Exemple

En visió artificial, una imatge pot estar representada per milers de valors numèrics. Reduir aquesta informació a 2 o 3 dimensions permet: visualitzar patrons, accelerar altres models, eliminar soroll

Analogia

Fer un resum d'un llibre: no conté totes les frases, però conserva les idees principals.

Detecció d'anomalies

La detecció d'anomalies busca identificar dades que no segueixen el comportament habitual del conjunt.

Aquestes anomalies poden indicar: errors, frau, atacs, fallades de sistemes, casos excepcionals

Exemple

En una xarxa informàtica, si tots els ordinadors generen un tràfic similar i un en genera molt més de sobte, pot indicar una infecció o un atac.

Analògia

En una classe on gairebé tots els alumnes arriben entre les 8:00 i les 8:10, un alumne que arriba cada dia a les 9:30 és un comportament anòmal que cal investigar.

*En no supervisat, **interpretar els resultats forma part essencial del treball**. El model pot crear clústers, però és el professional qui ha d'entendre què signifiquen i si són útils.*

En els models no supervisats, el preprocessament és crític. Decisions com escalar o no les dades, eliminar outliers, transformar variables categòriques,.. pot canviar radicalment el resultat trobat. Per exemple si una variable té molt més grans que les altres, dominarà les distàncies i distorsionarà el model.

Clustering (Agrupament)

El *clustering* (agrupament) és una de les tècniques més importants i utilitzades del Machine Learning no supervisat. La seva tasca és identificar instàncies similars i assignar-les a grups d'instàncies similars.

És important entendre que:

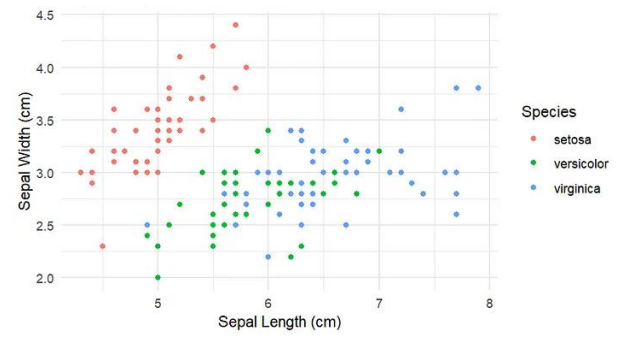
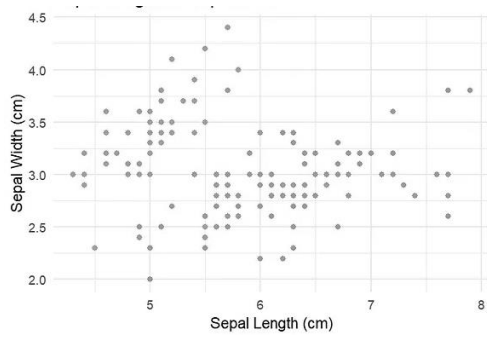
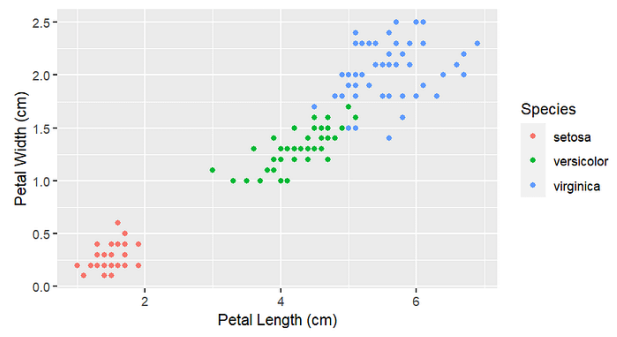
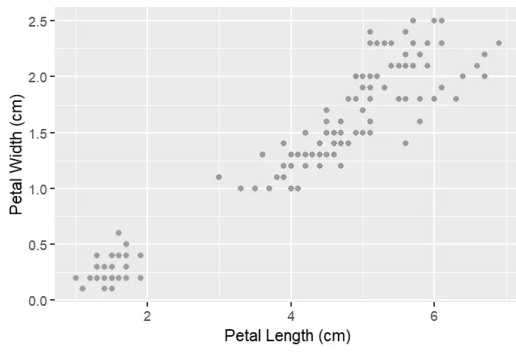
- El model no coneix prèviament els grups
- No hi ha etiquetes
- No hi ha una “resposta correcta”

El clustering no serveix per predir, sinó per descobrir estructura.

En un procés de classificació també assignem una instància a un grup, però a diferència de la classificació, l'agrupament és una tasca no supervisada.

En l'exemple si partim del dataset Iris a on tenim les característiques de `petal_width` respecte el `petal_length` tindrem el següent diagrama de dispersió que a simple vista podem distingir només dos grups de flors, però si mirem la figura del costat dret veiem que existeixen 3 grups de flors iris. Però si utilitzem les altres dos característiques addicionals de `sepal_width` i `petal_length` els models no supervisats poden aprofitar bé totes les característiques i poder identificar els 3 grups bastant bé.



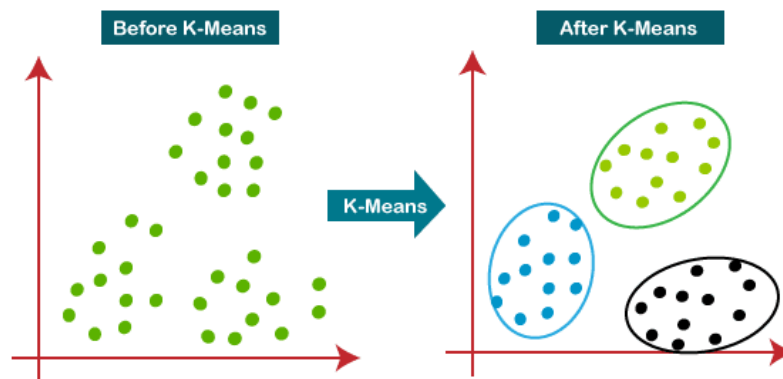


K-Means

K-Means és un algorisme de *clustering*. El seu objectiu és dividir les dades en K grups (clústers) de maners que les dades dins d'un mateix grup siguin molt semblants i les dades de grups diferents siguin poc semblants.

Aquest algorisme és ràpid i eficient. Sovint amb poques iteracions convergeix.

Va ser proposat per Stuart Lloyd a Bell Labs (1957), però es va [publicar](#) fora l'empresa el 1982.

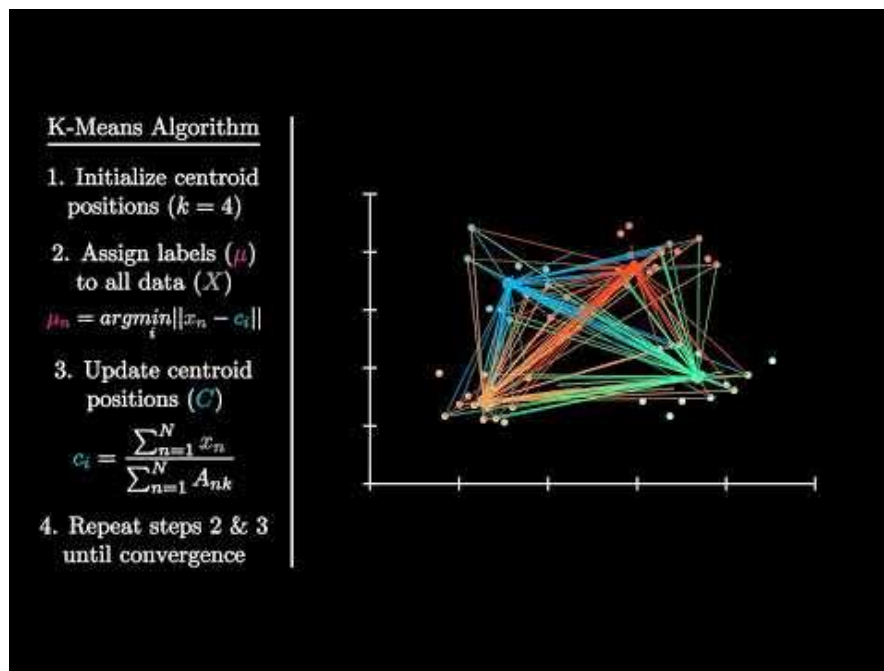
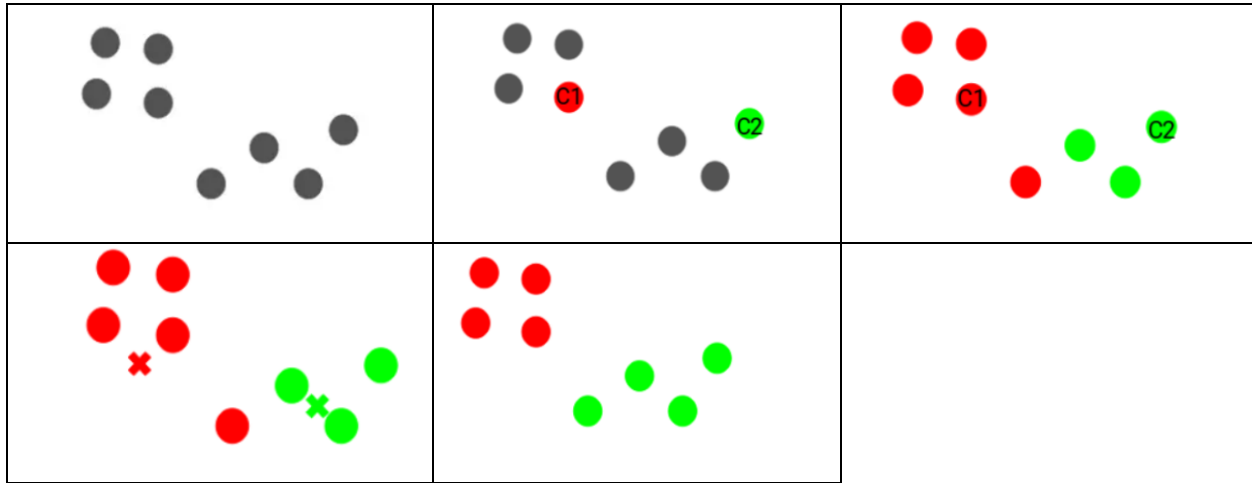


Com funciona?

- **Aleatòriament escollim k punts** del conjunt de dades. Aquests punts actuaran com a centroides inicials del clústers.
- **Iterem fins que convergim** (els centroides no canvien de manera significativa) o quan s'arriba a un **nombre determinat** d'iteracions.
 - **Assignació:** Cada punt de dades l'assignem al centroide més proper.
 - **Actualització de centroides:** Un cop hem assignat tots els punts de dades a un clúster. Recalculem els centroides prenent la mitjana de tots els punts de dades assignats a cada clúster.

Està garantitzat que convergeix perquè la distància quadràtica mitjana entre les instàncies i els seus centroides més propers només poden reduir-se a cada pas, i com que no pot ser negatiu, garantim que convergirà.

Exemples visuals



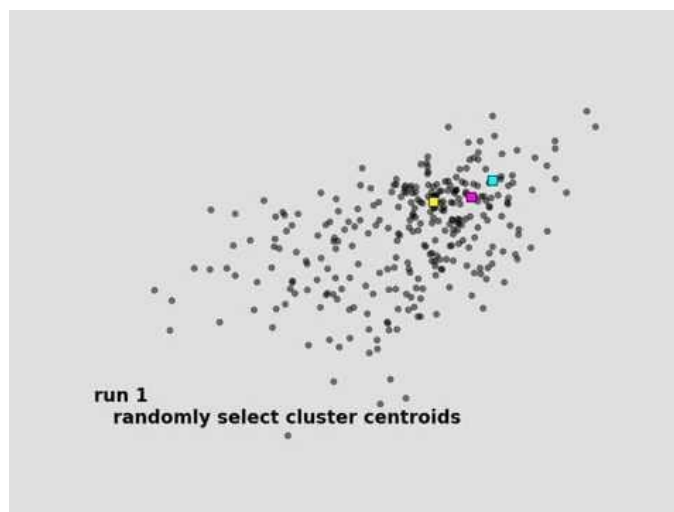
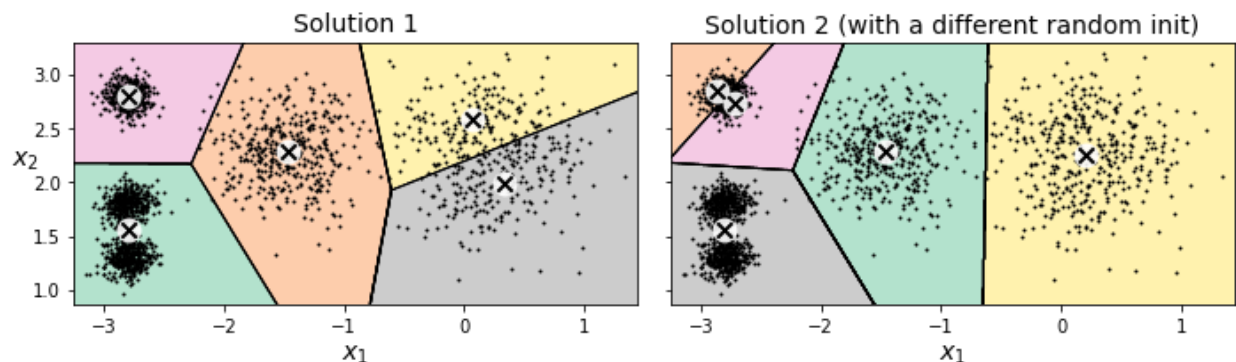
https://youtu.be/2lZZ_FzIJY

Inicialització dels centroides

K-Means és un algorisme heurístic perquè utilitza una estratègia pràctica i eficient per trobar una **solució bona**, però **no garanteix** trobar la **millor solució** possible.

Encara que està garantit que l'algorisme convergeixi, però no té perquè ser una solució adequada. Que ho faci o no dependrà de la inicialització del centroides.

Per exemple, en aquestes dues solucions veiem que totes dues tenen resultats diferents i no òptims degut a una mala inicialització



<https://www.youtube.com/watch?v=9nKfViAfaiY>

En el vídeo veiem que en les 4 execucions que es fa sobre el mateix conjunt de dades tenim 4 resultats diferents.

Si tenim una idea aproximada a on han d'estar els centroides amb Scikit-Learn podem establir l'hiperparàmetre `init` com una matriu NumPy a on cal també establir `n_init = 1`.

```
centroids = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])
kmeans = KMeans(n_clusters=5, init=centroids, n_init=1, random_state=42)
kmeans.fit(X)
```

`n_init`: serveix per indicar a l'algorisme que s'executi `n_init` vegades i que es quedi amb la millor solució. S'utilitza una mètrica per saber si una solució és millor que l'altre. Aquesta mètrica s'anomena inèrcia (*inertia*) i és la suma de les distàncies quadrades dels punts al seu centroide.

La classe `KMeans` de Scikit-Learn executa el model `n_init` vegades i manté el model amb la inèrcia més baixa.

Podem consultar aquest valor mitjançant la propietat `inertia_`

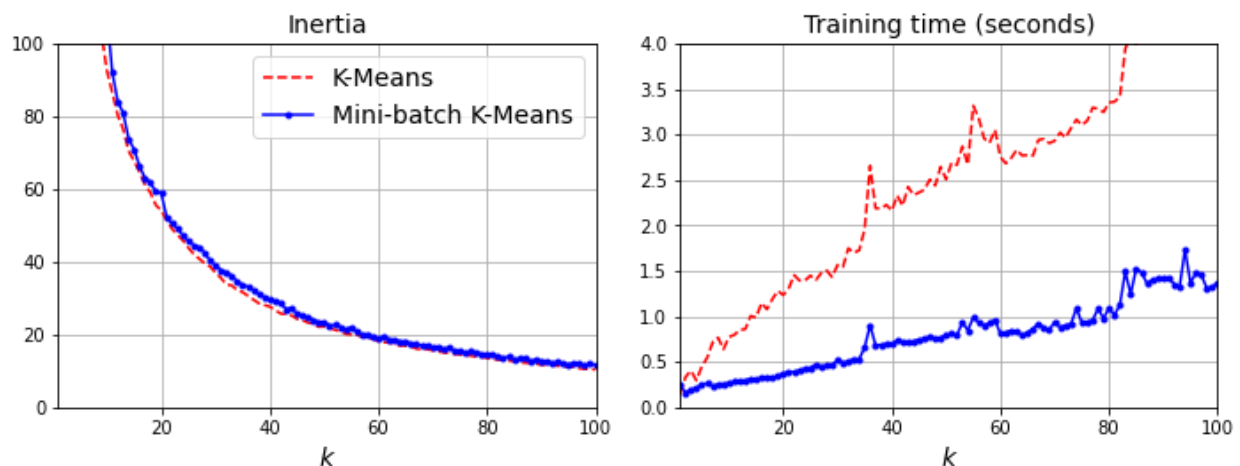
```
kmeans = KMeans(n_clusters=5, init=centroides, n_init=1, random_state=42)
kmeans.fit(X)
kmeans.inertia_
```

L'any 2006, David Arthur i Sergei Vassilvitskii van desenvolupar un enfocament nou i molt més eficient per al procés d'inicialització dels centròides. Van [publicar](#) la seva proposta el 2007 amb el nom de **k-means++**. El que fa aquesta millor és que els centròides inicials siguin punts que estiguin allunyats entre si. Cal dir que la classe `KMeans` de Scikit-Learn ja utilitza aquest mètode d'inicialització de centròides per defecte.

Una altra proposta de millora a l'algorisme k-means va ser la proposada a l'[article](#) publicat per Charles Elkan l'any 2003 a on amb conjunts de dades grans amb molts grups, l'algorisme es pot accelerar per evitar molts càlculs de distàncies innecessàries.

No sempre accelera l'entrenament, i a vegades l'empitjora, però es pot provar utilitzant l'hiperparàmetre `algorithm='elkan'`. Per defecte `algorithm='lloyd'`

Una altra variant important del k-means va ser proposat a l'[article](#) publicat l'any 2010 per David Sculley en que proposava de no utilitzar tot el conjunt de dades complet a cada iteració, sinó d'utilitzar minilots. Això permet accelerar de manera general l'algorisme 3 o 4 vegades, però pot empitjorar la seva inèrcia.



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Aurélien Géron

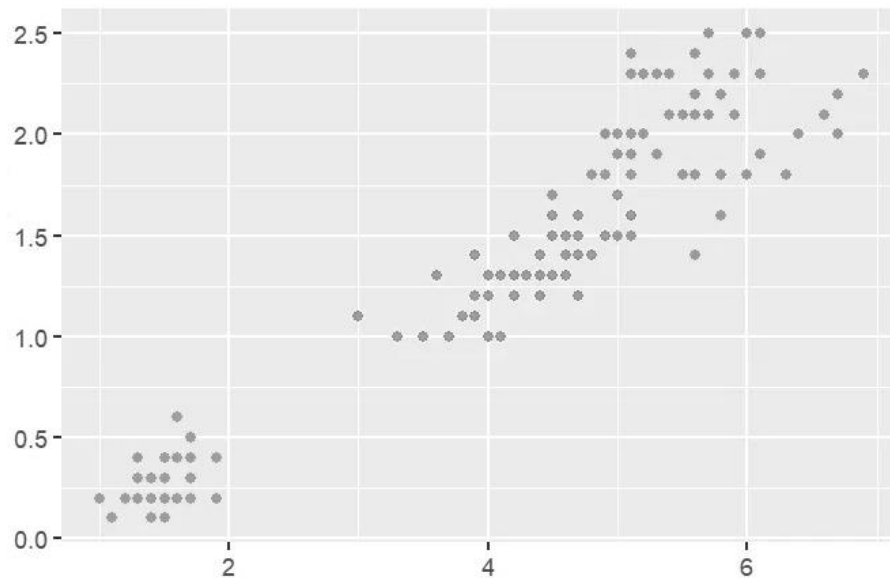
```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5, n_init=3, random_state=42)
minibatch_kmeans.fit(X)
```

Número de clústers (k)?

Un altre punt important i que segurament us heu preguntat és quin valor de k hem d'escollir?

Per exemple en el següent núvol de punts quin és la k a escollir $k=2$ o $k=3$?



Podríem pensar en ajudar-nos de la **inèrcia** per escollir el valor de k , però això **no és una bona idea** perquè a mesura que incrementem k la inèrcia anirà baixant.

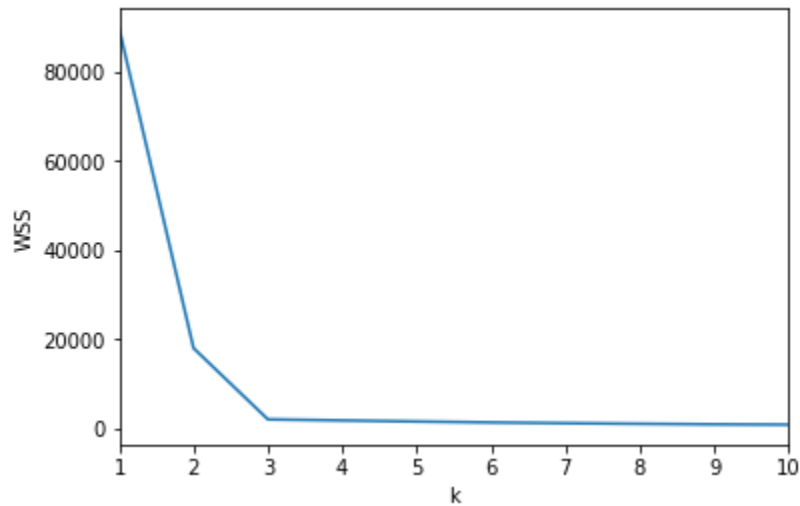
En aquest apartat mirarem dos mètodes que poden ser útils per trobar aquesta k .

Mètode del colze (*Elbow method*)

Aquest mètode és el més conegut per determinar el nombre òptim de clústers.

Bàsicament el que fa és calcular la suma d'errors quadràtics dins del clúster (*Within-Cluster Sum of Squared Errors*, WSS) per a diferents valors de k , i tria la k per la qual el WSS comença per primera vegada a disminuir menys.

En el següent gràfic podem veure que a **$k=3$ hi ha un colze** a on la gràfica comença a aplanar-se



En el següent fragment de codi proporcionat per [@khyatimahendru](#) podem veure com calcular aquest WSS per cada valor de K.

```
from sklearn.cluster import KMeans

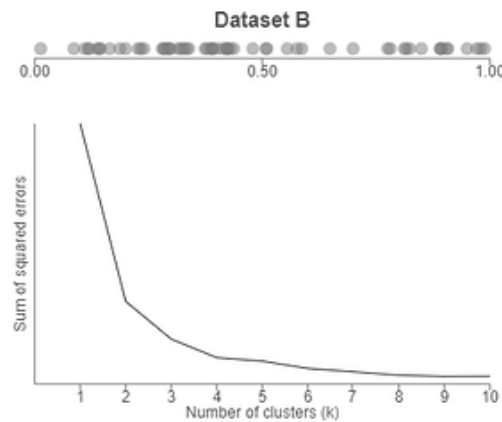
# function returns WSS score for k values from 1 to kmax
def calculate_WSS(points, kmax):
    sse = []
    for k in range(1, kmax+1):
        kmeans = KMeans(n_clusters = k).fit(points)
        centroids = kmeans.cluster_centers_
        pred_clusters = kmeans.predict(points)
        curr_sse = 0

        # calculate square of Euclidean distance of each point from its cluster center and
        # add to current WSS
        for i in range(len(points)):
            curr_center = centroids[pred_clusters[i]]
            curr_sse += (points[i, 0] - curr_center[0]) ** 2 + (points[i, 1] - curr_center[1])
            ** 2

        sse.append(curr_sse)
    return sse
```

Mètode de la silueta (Silhouette Method)

Encara que el mètode del colze pot semblar un bon mètode és una mica matusser ja que a vegades no ens és fàcil decidir tampoc el valor k perquè no veiem un colze molt clar en el gràfic. Un clar exemple el trobem en el següent gràfic, a on no tenim clar quin seria el valor de k. K=3 o k=4



Un mètode més precís, però també més car a nivell computacional és utilitzar la puntuació de la silueta (silhouette measure). Aquest valor ens indica fins a quin punt un punt és similar al seu propi clúster (cohesió) en comparació amb els altres (separació). Aquest varia entre -1 i 1:

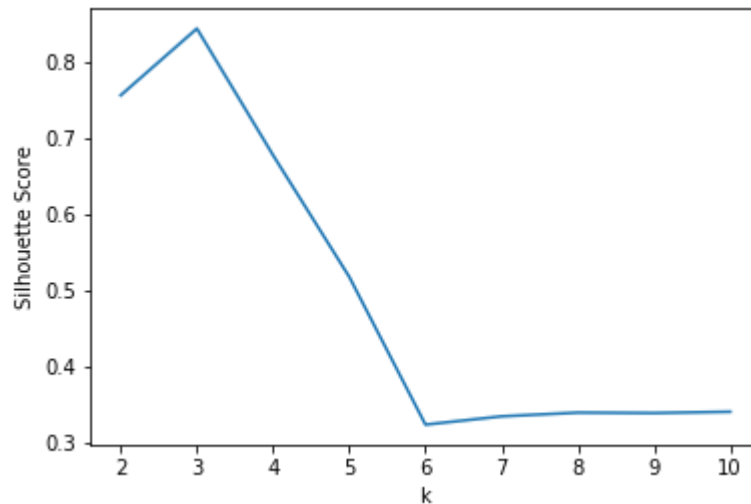
- Un coeficient proper a +1 significa que la instància està molt a dins del seu grup i lluny d'altres grups.
- Un coeficient proper a 0 significa que està a prop del límit d'un grup
- Un coeficient proper a -1 significa que la instància pot haver-se assignat a un grup equivocat.

La puntuació de silhouette es pot calcular fàcilment en Python utilitzant el mòdul `metrics` de Scikit-Learn.

```
from sklearn.metrics import silhouette_score

sil = []
kmax = 10

# La dissimilitud no es pot definir per a un únic clúster, per tant, mínim=2
for k in range(2, kmax+1):
    kmeans = KMeans(n_clusters = k).fit(x)
    labels = kmeans.labels_
    sil.append(silhouette_score(x, labels, metric = 'euclidean'))
```



Fer un notebook amb quest codi: <https://medium.com/analytics-vidhya/how-to-determine-the-optimal-k-for-k-means-708505d204eb>

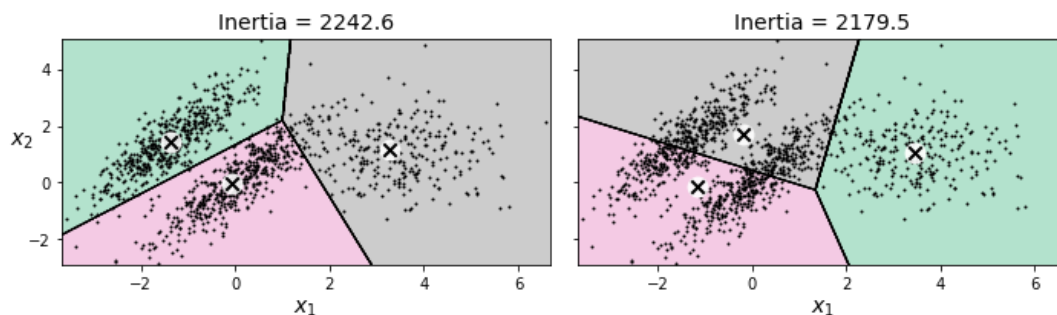
Límits de k-means

Tot i les seves virtuts, k-means no és perfecte com hem pogut veure. Hem d'executar l'algorisme varies vegades per evitar solucions no òptimes com també hem d'especificar a priori el número de clústers.

Un altre inconvenient de k-means és que no funciona molt bé quan els grups tenen tamanys diferents. Densitats diferents o formes no esfèriques.

Per exemple a la següent figura mostra com k-means agrupa un conjunt de dades que conté 3 grups el·lipsoïdals de diferents dimensions, densitats i orientacions.

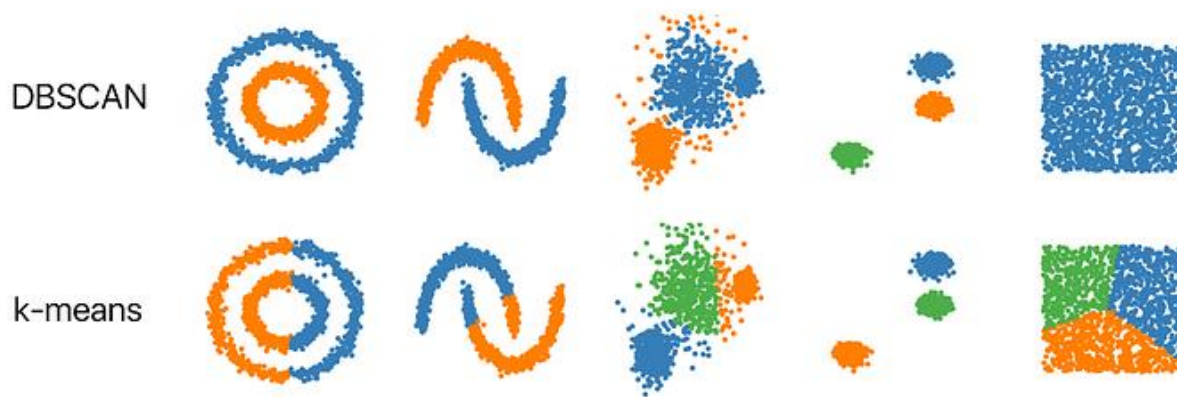
Tot i la primer solució ser bona el grup de la dreta agafa punts del centroide inferior. En aquests tipus de grups el·líptics funcionen molt bé els Gaussian Mixture Models.



És Important escalar les característiques d'entrada abans d'executar k-means ja que permet que els grups no s'estirin molt. Això no garanteix que tots els grups siguin esfèric, però ajuda molt al k-means

DBSCAN

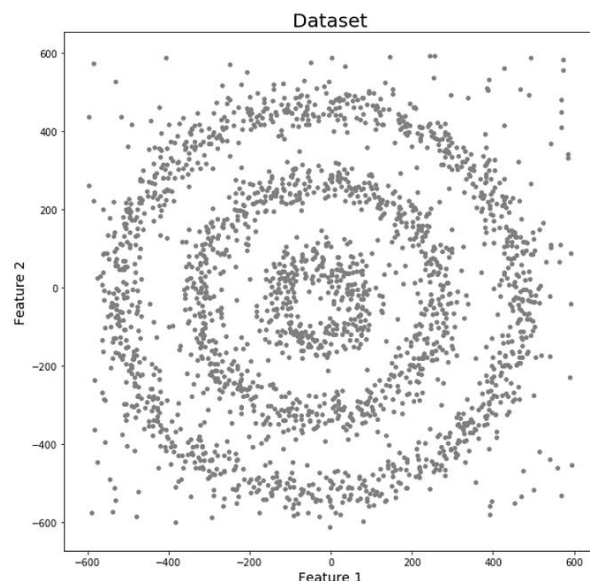
DBSCAN (**D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise) és un model d'aprenentatge no supervisat basat en densitats. A diferència de K-Means, **DBSCAN no necessita especificar el nombre de clústers ni utilitzar centroides**. La informació s'agrupa mitjançant àrees d'alta concentració de punts. És a dir els clústers es formen a partir de **zones on les dades estan molt concentrades**, i els punts aïllats es consideren **soroll o anomalies**.

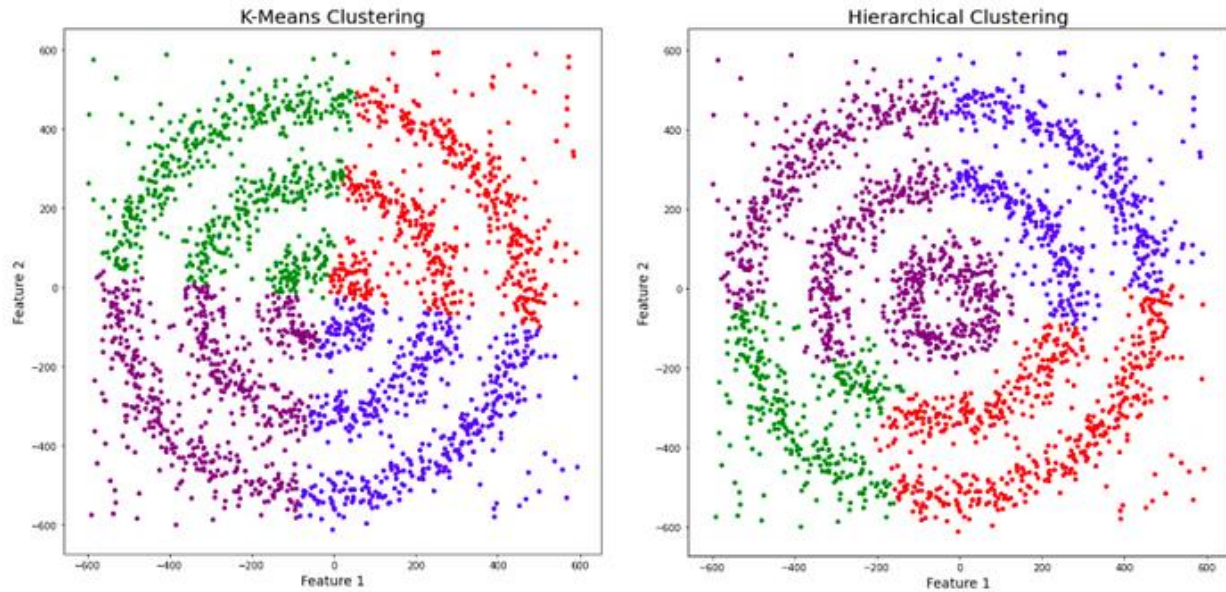


Un dels exemples més clars per entendre DBSCAN és mitjançant on tenim punts de dades densament distribuïts en forma de cercles concèntrics:

En aquest dataset podem veure tres clústers densos diferents amb una mica de soroll.

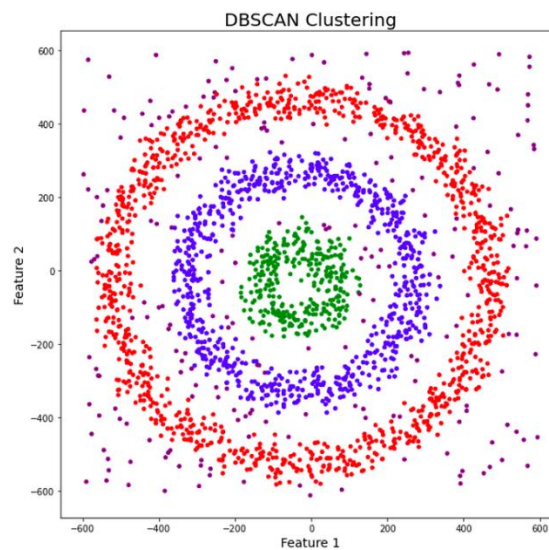
Si executem algorismes com K-Means o Clustering jeràrquic veiem que K-Means ha fet la seva feina i ha realitzat la seva agrupació mitjançant la distància dels diferents punts. S'han tractat les dades com un pastís on s'han fet 4 talls iguals.





Malauradament aquestes estratègies no funcionen amb aquests tipus de dades.

Si mirem la solució que ens dona DBSCAN veiem que no només ens agrupa les dades de forma correcte, sinó que també ens detecta el soroll (color morat)



DBSCAN agrupa punts **densament agrupats** en un mateix clúster i identifica clústers en grans conjunts espacials observant la densitat local. És robust als valors atípics i no requereix especificar el nombre de clústers prèviament (a diferència de K-Means).

Només necessita **epsilon** i **minPoints**.

DBSCAN crea un cercle de radi ϵ al voltant de cada punt i els classifica en nucli, frontera o soroll:

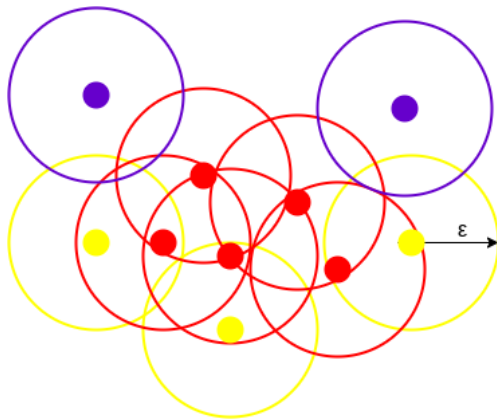
- Nucli: almenys `minPoints` dins el radi.
- Frontera: menys de `minPoints` però més d'un punt.
- Soroll: cap altre punt dins ϵ .

DBSCAN utilitza habitualment distància euclidiana (tot i que se'n poden usar d'altres) i només necessita un escaneig del conjunt de dades.

Com funciona:

- Per cada instància, l'algorisme compta quantes observacions es situen a una distància ϵ (**èpsilon**) d'ella. D'aquesta regió conjunt de punts s'anomena "**veïnatge ϵ** ".
- Si una instància té un mínim de `min_samples` exemplars dins el seu veïnatge (incloent-se a si mateixa), es considera una instància nucli o punt nucli (*core point*).
- Quan l'algorisme troba una instància nucli que encara no pertany a cap clúster, inicia un nou clúster a partir d'aquesta instància. Aquesta instància no és un centroid, sinó un punt inicial d'una zona d'alta densitat.
- El clúster es construeix per expansió: totes les observacions del veïnatge ϵ de la instància nucli s'afegeixen al clúster. Si alguna d'aquestes observacions també és una instància nucli, el procés d'expansió continua de manera recursiva.
- Les instàncies que no són nucli però que es troben dins del veïnatge ϵ d'una instància nucli es consideren instàncies frontera (*border points*). Aquestes instàncies pertanyen al clúster, però no poden fer-lo créixer.
- Qualsevol instància que no sigui una instància nucli o frontera i no tingui una en el seu veïnatge es considerarà una anomalia o soroll i no pertanyerà a cap clúster.

L'algorisme DBSCAN crea un cercle de radi **epsilon (ϵ)** al voltant de cada punt de dades i els classifica com a **punt nucli (core point)**, **punt frontera** o **soroll**. Un punt de dades és un punt nucli si el cercle del seu voltant conté com a mínim **minPoints** punts. Si el nombre de punts és inferior a **minPoints**, es classifica com a **punt frontera**; i si no hi ha cap altre punt de dades dins del radi epsilon al voltant del punt, aquest es considera **soroll**.



Aquesta figura mostra un clúster creat per DBSCAN amb **minPoints = 3**. Aquí, es dibuixa un cercle de radi epsilon (ϵ) igual al voltant de cada punt de dades. Aquests dos paràmetres ajuden a crear clústers espacials.

Tots els punts de dades amb almenys 3 punts dins del cercle, incloent-se a si mateixos, es consideren punts nucli i es representen en vermell. Tots els punts amb menys de 3 però més d'1 punt dins del cercle, incloent-se a si mateixos, es consideren punts frontera i es representen en groc. Finalment, els punts de dades que no tenen cap altre punt dins del cercle a part d'ells mateixos es consideren soroll, representat pel color porpra.

Reachability (accessibilitat) i connectivitat

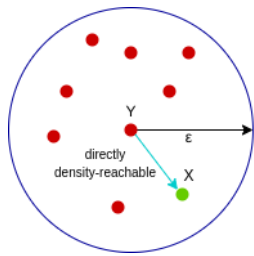
La reachability (accessibilitat) indica si un punt de dades pot ser accedit des d'un altre punt de manera directa o indirecta, mentre que la connectivitat indica si dos punts de dades pertanyen o no al mateix clúster. En termes d'accessibilitat i connectivitat, dos punts en DBSCAN es poden descriure com a:

- **Directament densitat-accessible (Directly Density-Reachable)**
- **Densitat-accessible (Density-Reachable)**
- **Densitat-connectat (Density-Connected)**

Directament densitat-accessible (Directly Density-Reachable)

Un punt **X** és **directament densitat-accessible** des d'un punt **Y** respecte **d'epsilon** i **minPoints** si:

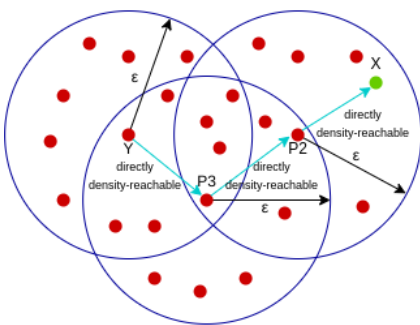
- X pertany al veïnatge de Y, és a dir, $\text{dist}(X, Y) \leq \text{epsilon}$
- Y és un punt nucli



Aquí, **X** és **directament densitat-accessible** des de **Y**, però el contrari/viceversa no és vàlid

Densitat-accessible (Density-Reachable)

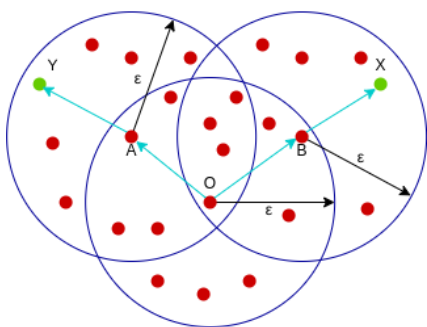
Un punt **X** és **densitat-accessible** des d'un punt **Y** respecte d'**epsilon** i **minPoints** si existeix una cadena de punts $p_1, p_2, p_3, \dots, p_n$ amb $p_1 = X$ i $p_n = Y$, tal que p_{i+1} és directament densitat-accessible des de p_i



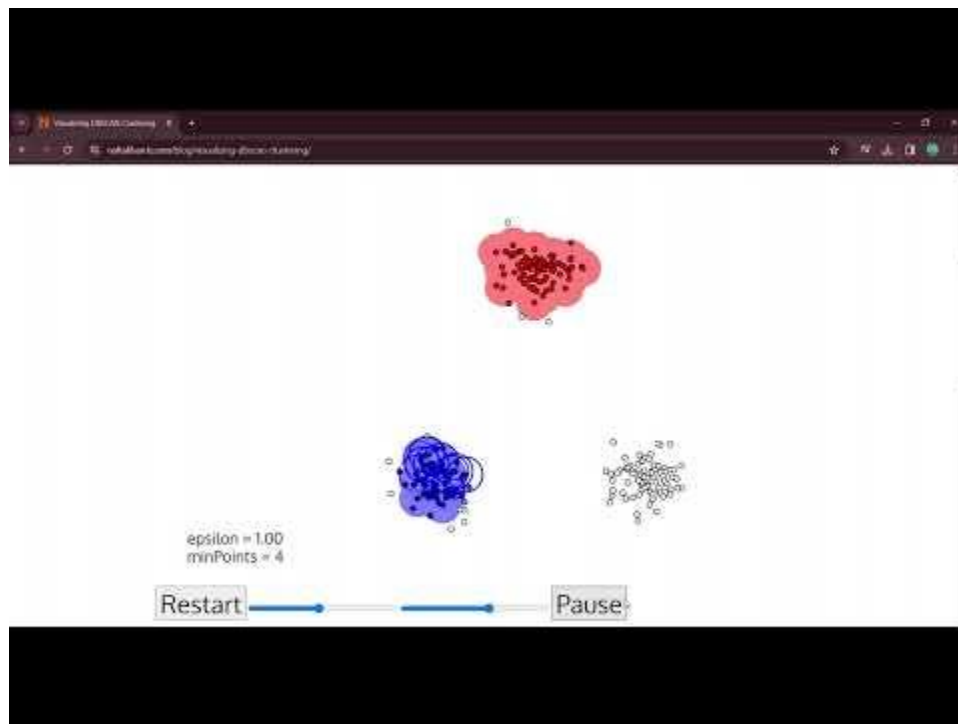
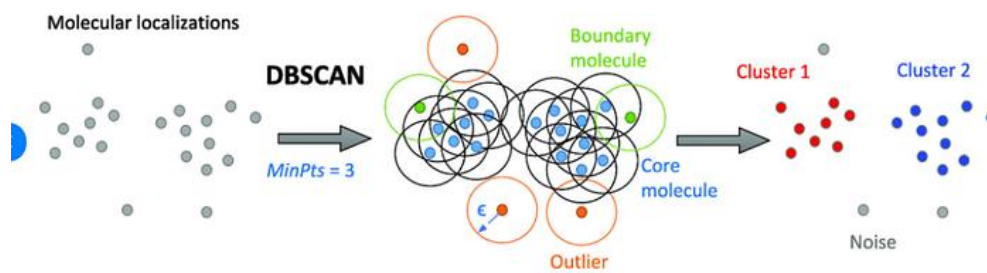
Aquí, **X** és **densitat-accessible** des de **Y**, ja que **X** és directament densitat-accessible des de **P2**, **P2** des de **P3** i **P3** des de **Y**. Tanmateix, l'invers no és vàlid.

Densitat-connectat (Density-Reachable)

Un punt **X** és densitat-connectat amb un punt **Y** respecte d'**epsilon** i **minPoints** si existeix un punt **O** tal que tant **X** com **Y** són densitat-accessibles des de **O** respecte d'**epsilon** i **minPoints**.



Aquí, tant **X** com **Y** són densitat-accessibles des de **O**; per tant, podem dir que **X** està densitat-connectat amb **Y**.



https://youtu.be/bKhZw0jvM_8

Un altre forma de definir el funcionament:

- Pas 1 — Identificar tots els punts com a punt nucli, punt frontera o punt soroll.
- Pas 2 — Per a tots els punts nucli que encara no estan agrupats:
 - Pas 2a — Crear un nou clúster.
 - Pas 2b — Afegir a aquest clúster tots els punts que no estiguin agrupats i que estiguin connectats per densitat amb el punt actual.
- Pas 3 — Per a cada punt frontera que encara no estigui agrupat, assignar-lo al clúster del punt nucli més proper.
- Pas 4 — Deixar tots els punts de soroll tal com estan

Selecció de paràmetres en DBSCAN

DBSCAN és molt sensible als valors **d'epsilon** i **minPoints**. Per això, és molt important entendre com seleccionar aquests valors. Una petita variació pot canviar significativament els resultats produïts per l'algorisme DBSCAN.

No té sentit prendre **minPoints = 1**, perquè **cada punt seria un clúster independent**. Per tant, ha de ser com a **mínim 3**. Generalment, es pren com el doble del nombre de dimensions, tot i que el coneixement del domini també influeix en la seva elecció.

$$\text{minPoints} \geq \text{Dimensions} + 1$$

El valor **d'epsilon** determina la densitat mínima necessària per formar els clústers:

- ϵ massa petit $\rightarrow$ veïnatges petits $\rightarrow$ pocs punts arriben a ser "nucli" $\rightarrow$ molts punts queden com soroll, i surten clústers petits o fragmentats.
- ϵ massa gran $\rightarrow$ veïnatges massa grans $\rightarrow$ punts de grups diferents es connecten $\rightarrow$ clústers es fusionen i es perd estructura

es pot decidir a partir del gràfic de K-distància (*K-distance graph*). El punt de màxima curvatura (el colze) d'aquest gràfic indica el valor adequat d'epsilon. Si el valor d'epsilon és massa petit, es crearan massa clústers i molts punts es classificaran com a soroll. En canvi, si és massa gran, diversos clústers petits es fusionaran en un de sol i es perdrà detall.

Els desavantatges més importants de DBSCAN són:

- Escollir bé els hiperparàmetres
- DBSCAN té dificultats amb clústers de densitats variables. Pot fallar a l'hora de connectar regions amb una densitat de punts més baixa amb la resta del clúster, cosa que pot donar lloc a assignacions de clústers subòptimes en conjunts de dades amb regions de densitat desigual.



GMM (Gaussian Mixture Models)

En altres models com K-means o DBSCAN, assignem cada punt de dades a un únic clúster. És a dir, cada observació pertany al clúster A, B o queda fora.

Però com s'ha vist o tenim amb molts problemes reals aquesta visió és massa rígida ja que tenim grups que es solapen, les fronteres entre grups no són clares o un element pot tenir característiques de més d'un grup.

Un **Gaussian Mixture Model** és un model de Machine Learning no supervisat que assumeix que **les dades observades provenen de la combinació (*mixture*) de diverses distribucions gaussianes**.

A diferència de K-Means, que assigna cada punt de dades a un clúster concret, GMM permet que un punt de dades pertanyi a diferents clústers amb diferents probabilitats.

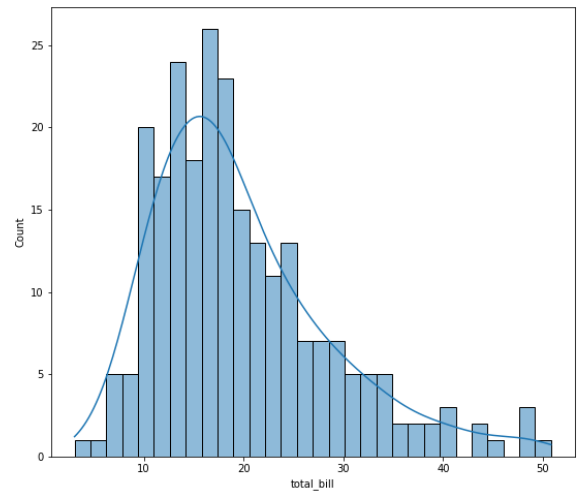
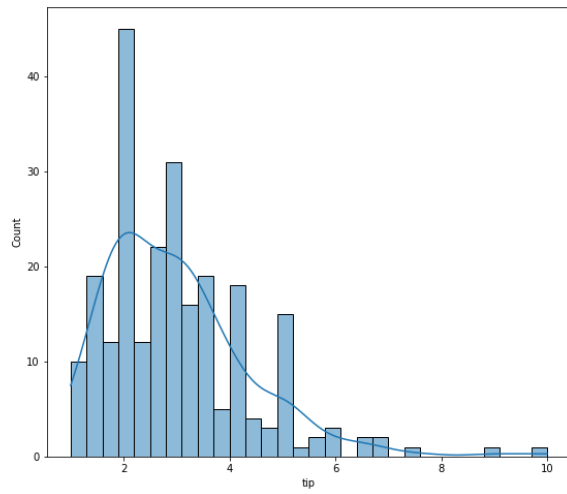
Igual que l'algorisme K-Means, el model **GMM ha de començar des d'algun punt**, utilitzant **paràmetres inicials aleatoris i construint el model a partir d'aquí**. Un cop triats aquests paràmetres inicials, l'algorisme inicia una sèrie de càlculs per trobar els **millors pesos i mitjanes** necessaris per fer convergir les distribucions gaussianes estimades cap a les definides pels paràmetres inicials. En altres paraules, decideix sota quina distribució cau cada punt de dades.

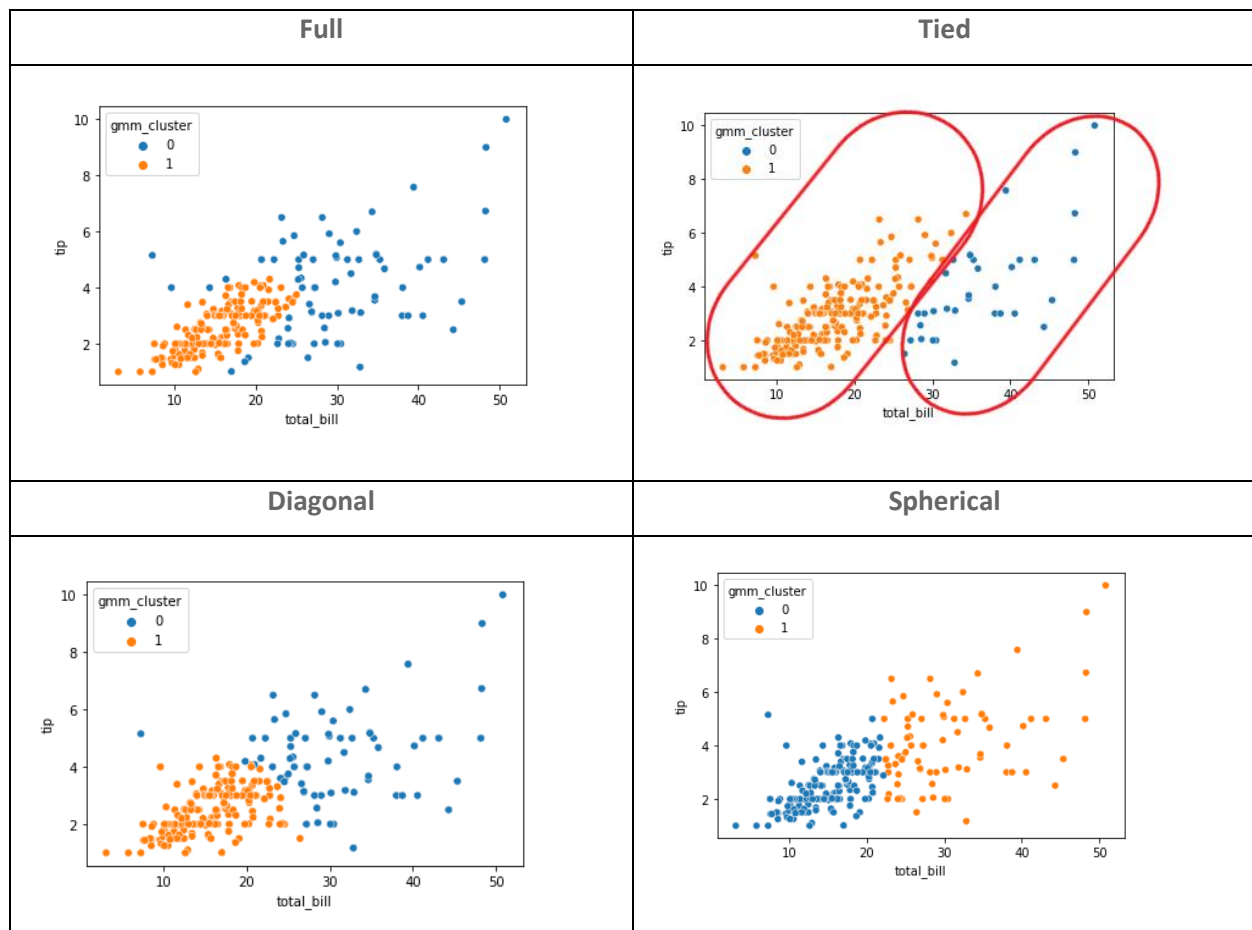
Per a això, existeix un hiperparàmetre anomenat **init_params**, on pots escollir entre punts inicials aleatoris (`random`) o utilitzar el valor per defecte (`kmeans`). Utilitzar K-Means pot ajudar que la convergència sigui més ràpida, així que recomano deixar l'opció per defecte.

Un altre paràmetre interessant és `covariance_type`, que permet triar entre `full`, `tied`, `diag` i `spherical`:

- **Full**: Cada component pot adoptar de manera independent qualsevol posició i forma.
- **Tied**: Tots els components tenen la mateixa forma, però aquesta pot ser qualsevol.
- **Diagonal**: Els eixos del contorn estan alineats amb els eixos de coordenades, però les excentricitats poden variar entre components.
- **Spherical**: és un cas "diagonal" amb contorns circulars (esfèrics en dimensions superiors, d'aquí el nom).

Anem a veure un exemple utilitzant diferents covariàncies amb el dataset de Tips ([Kaggle](#)) i les variables `total_vill` i `tip`.





- **Full:** Veiem formes totalment diferents en els dos clústers
- **Tied:** Els clústers tenen la mateixa forma. Viem que ens fa dos formes força el·lipsoidals.
- **Diagonal:** Aquí, la forma segueix els eixos de coordenades
- **Spherical:** És el resultat més semblant a K-Means, ja que els clústers es defineixen amb una forma esfèrica).

El GMM pot adoptar formes més complexes i el·líptiques que K-Means no pot, però també cal dir que és un model computacionalment més intens ja que requereix de més recursos i temps de càlcul.

RESUM

Característica	K-Means	DBSCAN	GMM
Tipus	Distància	Densitat	Probabilístic
Cal K	Sí	No	Sí
Forma clústers	Rodons	Arbitrària	El·líptics
Detecta outliers	✗	✓	✗
Assignació suau	✗	✗	✓
Escala bé	✓	⚠	⚠
Interpretació	Simple	Geomètrica	Estadística
Ideal per	Dades netes	Dades reals	Dades solapades

K-Medoids (PAM)

HDBSCAN

DTW (Dynamic Time Warping)

K-Means per seqüències

ANNEX

Reducció de la dimensionalitat

PCA

t-SNE

UMAP

Autoencoders (introducció molt bàsica)

REFERÈNCIES

- Aurélien Géron (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- Continguts de la docent Cristina Gómez de IES de Teis Vigo
- <https://medium.com/@amitvsolutions/the-complete-unsupervised-learning-handbook-ml-algorithms-0bc9d91bc7a8>
- <https://medium.com/@abhaysingh71711/k-means-clustering-a-deep-dive-into-unsupervised-learning-81213f56cfc9>
- <https://towardsdatascience.com/breaking-it-down-k-means-clustering-e0ef0168688d/>
- <https://medium.datadriveninvestor.com/mastering-k-means-clustering-a-step-by-step-guide-de72e66377a6>
- <https://blog.gopenai.com/mastering-k-means-clustering-a-deep-dive-with-code-and-visuals->

[b2d15e542281](#)

- <https://medium.com/analytics-vidhya/how-to-determine-the-optimal-k-for-k-means-708505d204eb>
- <https://antonio-richaud.com/blog/archivo/publicaciones/36-kmeans-vs-gmm.html>
- K-Means: <https://towardsdatascience.com/k-means-clustering-algorithm-applications-evaluation-methods-and-drawbacks-aa03e644b48a/>
- GMM: <https://towardsdatascience.com/a-simple-introduction-to-gaussian-mixture-model-gmm-f9fe501eef99/>
- GMM: <https://towardsdatascience.com/gaussian-mixture-models-for-clustering-3f62d0da675/>
- <https://antonio-richaud.com/blog/archivo/publicaciones/36-kmeans-vs-gmm.html>
- DBSCAN: <https://medium.com/@abhaysingh71711/dbscan-explained-unleashing-the-power-of-density-based-clustering-72a51ba40fdf>
- DBSCAN: <https://medium.com/@sachinsoni600517/clustering-like-a-pro-a-beginners-guide-to-dbscan-6c8274c362c4>